Exceptions

CSCI 1101

2024-04-02

What Could Go Wrong?

Unfortunately, there are many things that can go wrong in a program:

```
"hello" + 1
```

```
undefined_variable += 1
```

```
[42][1]
```

```
42 / 0
```

```
int('forty two')
```

Some can be avoided by **static analysis** (analyzing a program without running it).

Others can be detected only by running the program to see what happens.

Exceptions

The things returned by python when something goes wrong are **exceptions**.

These are just instances of certain types that are not the normal result of a computation.

Examples include:

- `TypeError` when something has the wrong type,
- `NameError` when you refer to a variable that is not defined,
- `IndexError` when a list index is out of bounds,
- `ZeroDivisionError` when you try to divide by zero,
- `ValueError` when something has the right type but not a valid value.

Proceeding with Caution

One way to avoid errors is to use conditionals and only try to do things that we know will succeed.

```
def is_int_string(text) :
    '''
    sig: (str) --> bool
    determines if a string is parseable as an int
    '''
    return str.isdigit(text) or \
    (text[0:1] == '-' and str.isdigit(text[1:]))
```

Proceeding with Caution (ctd.)

```
def cautious_interactive_division() :
    x_str = input('please enter the numerator: ')
    y_str = input('please enter the denominator: ')
    if is_int_string(x_str) and is_int_string(y_str) :
        # it's safe to parse ints from them
        (x_int , y_int) = (int(x_str) , int(y_str))
        if y_int != 0 :
            # it's safe to divide them
            result = x_int / y_int
            print(f"{x_str} / {y_str} = {result}")
        else :
            print("you can't divide by zero")
    else :
        print('to divide you must enter two numbers')
```

Ploughing Ahead

Sometimes it's easier to ask forgiveness than permission.

```
def brash_interactive_division() :
    x_str = input('please enter the numerator: ')
    y_str = input('please enter the denominator: ')
    try :
        x_int = int(x_str)     # possible ValueError
        y_int = int(y_str)     # possible ValueError
        result = x_int / y_int # possible ZeroDivisionError
        print(f"{x_str} / {y_str} = {result}")
    except ZeroDivisionError :
        print("you can't divide by zero")
    except ValueError :
        print('to divide you must enter two numbers')
```

This style can make programs more clear and concise by focusing on the main execution path.

Exception Handling

Exceptions introduce a new flow-control construct, the **try statement**:

```
try :
    <try_block>
except <exception_guard_1> :
    <except_block_1>
 ⋮
except <exception_guard_n> :
    <except_block_n>
```

The try statement has two parts: a **try clause** and one or more **except clauses**.

Each of these clauses contains a code **block** (nonempty sequence of statements).

The except clauses each have a **guard**, which is an exception type.

Exception Handling (ctd.)

When python encounters a `try` statement, the following happens:

- statements in the `try` clause are executed as normal,
- if no exceptions occur in the `try` clause, then all `except` clauses are skipped,
- if an exception does occur in the `try` clause, then control passes immediately to the `except` clauses,
- the guards of the `except` clauses are tested in order for the first match,
- if a match is found then its block is executed and the exception is **handled**,
- if no match is found then the exception is **unhandled**.

An `except` clause guarded by multiple exception types can handle any of them.

An `except` clause guarded by no exception types can handle any exception.

Example: Printing Some

A dictionary containing sets of day names indexed by start letter:

```
# sig: dict (str , set str)
day_dict = {
  'm':{'Monday'}, 'w':{'Wednesday'}, 'f':{'Friday'}, 'b':{},
  't':{'Tuesday','Thurdsday'}, 's':{'Saturday','Sunday'}}
```

A function that prints some value associated with a given key (if possible):

```
def print_some (dictionary , key) :
    ''' sig: (dict (a , set b) , a) --> NoneType '''
    try :
        candidates = dictionary[key]  # possible KeyError
        result = list(candidates)[0]  # possible IndexError
        print(f"{result}")
    except (KeyError , IndexError) :
        print("No can do")
```

Activity: Average

Write a function

```
average : (list int) --> float
```

that returns the average (mean) of a list of numbers, or 0.0 if the list is empty.

Do this using exceptions and *without* using any conditionals.

For example:

```
> average([1, 2, 3, 4])
2.5
> average([])
0.0
```

To Do

Finish *Homework 8: Sets and Dictionaries* on CodeRunner by 11:00AM this Thursday.

Finish *Project 3 - Automated Grader (Part 1)* on CodeRunner by midnight this Friday.

Study for midterm 2 next Tuesday (2024-04-09).